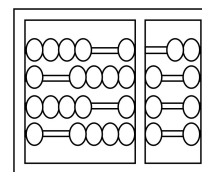




INF0093 - PROJETO
PRÁTICO COM
SISTEMAS
MULTIAGENTES - 2S
2026



PROF. MARCELO DA SILVA
REIS
msreis@unicamp.br

Entregável 4: Robustez, Ética e Avaliação Final

Campinas, 18 de setembro de 2026

Sumário

1	Objetivo	2
2	Formato	2
2.1	Estrutura herdada	2
3	Robustez e confiabilidade	2
3.1	Contenção	3
3.2	Falhas silenciosas	3
3.3	Confiabilidade	3
4	Comparação arquitetural	4
4.1	Sinal e ruído	4
5	Avaliação ética	4
5.1	Assimetria entre erros e sua distribuição	4
5.2	Rastreabilidade	6
5.3	Riscos próprios de arquiteturas multiagentes	6
5.4	Matriz de consequências	6
6	Recomendação final	7
7	Entrega do notebook	7
8	Critérios de avaliação	7

1 Objetivo

O quarto e último entregável é dividido em três partes: tornar o sistema resistente a falhas, medir a variação entre execuções, e comparar as versões construídas ao longo do curso de forma honesta, incluindo a avaliação de quem é prejudicado (e como) quando o sistema erra, possivelmente detectando e mitigando vieses discriminatórios.

Cabe aqui uma recomendação honesta: uma conclusão de que a arquitetura final não compensa é um resultado legítimo (e recorrente na prática). O que se avalia aqui é o processo de desenvolvimento e comparação das soluções, ou seja, é a qualidade da investigação: se as afirmações estão sustentadas por evidência, se as ressalvas foram declaradas, e se as decisões de projeto foram justificadas.

2 Formato

- Grupos de 1–3 estudantes, mantendo a composição das entregas anteriores.
- Entrega em notebook Jupyter (`.ipynb`), com saídas salvas.
- Documentação em Markdown e código em Python.
- O notebook deve conter a comparação das quatro versões.

Vale a mesma convenção das entregas anteriores: cada entrega é um notebook novo, que abre com a estrutura herdada e traz apenas o que é próprio daquela versão. Permanece também a alternativa de desenvolvimento em repositório GitHub, desde que o notebook continue sendo a entrega no Classroom, traga as instruções de execução do código do repositório, e o grupo gere uma *release* antes do término do prazo, informando a *tag* no notebook.

2.1 Estrutura herdada

Devem ser reaproveitados sem alteração o conjunto de casos congelado, as funções de verificação e o formato de saída. Os resultados das versões anteriores devem estar disponíveis, seja pelos arquivos salvos nos entregáveis anteriores, seja por reexecução. Se alguma versão anterior for reexecutada agora, use o mesmo modelo e o mesmo ambiente das demais, e diga no notebook quais números vieram de execução nova e quais vieram de arquivo antigo.

3 Robustez e confiabilidade

O sistema deve tratar as falhas que pode encontrar. Não é necessário cobrir todas: escolha as plausíveis para o seu problema e justifique as escolhas.

3.1 Contenção

Implemente e demonstre pelo menos três dos mecanismos abaixo:

- repetição com espera crescente e teto de tentativas, aplicada apenas a falhas transitórias;
- tempo limite por operação, com o esgotamento registrado no trace;
- degradação graciosa (entrega de resultado parcial em vez de nenhum);
- validação de saída, com tratamento definido para o caso de o formato quebrar;
- limite de passos no laço do agente, com registro quando for atingido.

Uma resposta produzida em modo degradado precisa se identificar como tal, tanto no trace quanto para quem a recebe.

3.2 Falhas silenciosas

Falha ruidosa interrompe a execução; falha silenciosa produz uma resposta errada sem qualquer sinal. A segunda é mais perigosa e exige verificação ativa. Implemente ao menos duas verificações que convertam falha silenciosa em falha detectável. Exemplos incluem conferir se:

- a evidência citada existe na fonte;
- a resposta permanece dentro do escopo declarado;
- a confiança declarada é compatível com a evidência apresentada.

Para demonstrar que a contenção funciona, recomenda-se induzir falhas deliberadamente, por exemplo tornando uma ferramenta instável com probabilidade conhecida (como a probabilidade de falha de 40% aplicada na Atividade Prática 7).

3.3 Confiabilidade

Execute o conjunto congelado **pelo menos três vezes** na versão final e relate:

- a taxa de acerto de cada execução;
- a faixa observada entre a menor e a maior taxa;
- quais casos mudaram de resultado entre execuções.

Relate a faixa, e não apenas a melhor rodada. Os casos instáveis (ora passam, ora não) merecem comentário: eles marcam a fronteira da capacidade do sistema, e costumam ser mais informativos que os casos que sempre passam. Lembre-se de que temperatura zero reduz a variação, mas não garante saídas idênticas em serviços de inferência distribuída como o oferecido pelo Groq.

4 Comparação arquitetural

Consolide os resultados das quatro versões em uma única tabela, contendo por versão:

- acertos em contagem e em proporção;
- faixa observada entre execuções, quando aplicável;
- chamadas ao modelo e a ferramentas;
- latência média e/ou mediana;
- custo estimado;
- modos de falha característicos.

Com esta comparação, podemos avaliar de forma apropriada o “trade off” entre desempenho e recursos empregados.

4.1 Sinal e ruído

Com conjuntos pequenos, diferenças agregadas raramente sustentam afirmações de superioridade. Calcule a faixa plausível de cada taxa (por exemplo, pelo intervalo de Wilson, ou então com mínimo-máximo) e indique quais pares de versões têm faixas sobrepostas. Quando as faixas se sobrepõem, a evidência forte deixa de ser a diferença agregada e passa a ser o caso específico que mudou de comportamento, acompanhado da explicação pelo trace. Prefira afirmações do tipo “a v3 passou a acertar o caso composto que a v2 errava, e o trace mostra que isso ocorreu porque a pergunta passou a ser dividida entre dois especialistas”.

5 Avaliação ética

Esta seção não é um texto filosófico/reflexivo. Ela é um conjunto de medições sobre danos, e cada afirmação precisa de evidência, como no restante do entregável.

5.1 Assimetria entre erros e sua distribuição

A taxa de acerto trata todos os erros como equivalentes, e eles não são. Atribua um peso de gravidade a cada tipo de erro do seu domínio, justificando cada peso pelo dano concreto que aquele erro causa, e recalcule a qualidade de cada versão de forma ponderada. Compare a ordenação das versões pela métrica simples e pela ponderada. Se a ponderação não alterar nada, verifique se os pesos foram atribuídos com convicção ou apenas preenchidos.

Um sistema com boa taxa geral pode falhar de forma concentrada. Separe os resultados por tipo de entrada e verifique se algum tipo destoa da média. Acrescente ao conjunto congelado pelo menos um caso que represente o perfil ou a situação menos usual do seu domínio, isto é, aquele com menor probabilidade de estar bem representado nos dados de treinamento do modelo. Um sistema que resolve ambiguidades e/ou falta de informação sempre contra o mesmo perfil não está sendo neutro.

Pares mínimos (detecção de possíveis vieses discriminatórios)

Separar os resultados por tipo de caso mostra onde o sistema erra mais, mas não mostra se ele trata pessoas diferentes de forma diferente. Para isso, construa ao menos um **par mínimo**: duas entradas idênticas, variando apenas um atributo que não deveria influenciar a resposta (por exemplo, “português culto vs. português informal”, “pesquisador vs. pesquisadora”). Se a fonte consultada pelo sistema nada diz sobre esse atributo, qualquer diferença entre as respostas foi produzida pelo sistema, e não pelos dados. Compare as duas variantes em mais de uma dimensão, porque o veredito raramente muda:

- a resposta está correta;
- a confiança declarada;
- a quantidade de evidência citada;
- o número de ressalvas e condicionais acrescentados.

É comum que o sistema responda corretamente às duas variantes e, ainda assim, entregue a uma delas uma resposta menos confiante, com menos evidência e mais condicionais. Esse tratamento desigual não aparece em nenhuma métrica de acurácia.

Antes de concluir que há viés, verifique duas coisas: se a diferença observada é maior que a variação da mesma variante entre execuções repetidas, e se ela se mantém ao reformular a pergunta. Um par mínimo executado uma vez não sustenta conclusão.

Ao interpretar o resultado, distinga duas explicações possíveis: o sistema é de fato pior naquele tipo de tarefa, ou aquele tipo tem poucos casos e o número não significa nada. Diga qual das duas o grupo defende, e com base em quê.

Sistemas que não interagem com humanos

Se o seu sistema não tem interação direta com humanos, de forma que o experimento de pares mínimos não faça sentido (por exemplo, um gerenciador de tráfego de veículos autônomos), substitua este experimento por uma explicação sobre possíveis maus usos de seu sistema, que levariam a violações éticas (declarar quais seriam).

5.2 Rastreabilidade

Verifique se o registro produzido permite responsabilização, e não apenas depuração durante o desenvolvimento:

- o registro sobrevive à execução, ou desaparece com a sessão;
- é possível identificar qual componente produziu cada afirmação;
- é possível mostrar ao usuário o trecho da fonte que sustenta a resposta;
- é possível reconstruir uma execução específica dias depois.

5.3 Riscos próprios de arquiteturas multiagentes

Discuta, com base no que observaram:

- **Difusão de responsabilidade:** quando vários agentes produzem uma resposta errada, nenhum deles errou por inteiro. O especialista respondeu corretamente dentro do seu escopo, o roteamento foi defensável, e a síntese omitiu um achado. A diluição da autoria do erro é uma propriedade da arquitetura, e não do modelo.
- **Consenso falso:** a agregação pode suprimir uma informação correta presente em apenas um dos componentes, produzindo confiança alta exatamente onde deveria haver dúvida. É o mesmo modo de falha que vocês já listam desde o Entregável 3, visto do ponto de vista do dano.
- **Confirmação humana:** em que ponto do fluxo o sistema deveria interromper e consultar o usuário. Considere ações irreversíveis, custo assimétrico do erro, confiança declarada baixa e discordância entre verificações. Exigir confirmação a cada passo não é segurança, é transferir o problema para o usuário.

5.4 Matriz de consequências

Produza uma tabela com uma linha por modo de falha observado, contendo o modo de falha, quem é prejudicado, a gravidade atribuída e a mitigação adotada. A matriz precisa ser coerente com os pesos da subseção sobre assimetria: se um modo de falha grave não corresponde a nenhum peso alto, ou a matriz está errada, ou a métrica está. Utilize como base a matriz apresentada na Atividade Prática 8.

6 Recomendação final

Ao final do notebook, indique qual versão o grupo levaria para uso real, informando:

- o critério que está sendo otimizado, seja qualidade, custo, latência ou previsibilidade;
- a evidência principal que sustenta a escolha;
- as ressalvas, isto é, o que os dados coletados não permitem afirmar;
- que evidência ainda faltaria, e como o grupo a obteria.

A recomendação pode ser uma versão intermediária, ou o próprio baseline. O que precisa estar sustentado é o raciocínio.

7 Entrega do notebook

A entrega do `.ipynb` deverá ser feita até dia **4 de outubro** (domingo), utilizando o Google Classroom da disciplina. O notebook deve ser executado do início ao fim e salvo com as saídas visíveis. Nenhuma chave de API deve estar escrita no notebook.

8 Critérios de avaliação

Critério	Peso
Tratamento de erros e conversão de falhas silenciosas	20%
Medição de confiabilidade e variação entre execuções	15%
Comparação arquitetural das quatro versões	20%
Honestidade estatística na leitura dos resultados	10%
Avaliação ética: pesos, pares mínimos, distribuição e matriz	20%
Recomendação final e ressalvas declaradas	10%
Documentação e reprodutibilidade	5%